

TerminalRPG — GUI Migration Plan

Project: TerminalRPG
Scope: Migrate from terminal-based UI (print/input) to a graphical user interface
Framework: tkinter (Phase 1–2), with Pygame upgrade path (Phase 3+)
Estimated Effort: 4–6 weeks (part-time)
Last Updated: 2025-07-02

Table of Contents

- 1. [Executive Summary](#)
- 2. [Current Architecture Analysis](#)
- 3. [Framework Decision](#)
- 4. [Migration Strategy](#)
- 5. [Files to Change — Complete Inventory](#)
- 6. [New Files to Create](#)
- 7. [Implementation Phases & Timeline](#)
- 8. [Detailed Phase Breakdown](#)
- 9. [Key Design Decisions](#)
- 10. [Save File Compatibility](#)
- 11. [Build System Updates](#)
- 12. [Testing Strategy](#)
- 13. [Risks & Mitigations](#)
- 14. [Appendix A: I/O Call Audit](#)
- 15. [Appendix B: GUI Wireframe Notes](#)

1. Executive Summary

TerminalRPG currently runs entirely through `print()` and `input()` calls across ~33 Python files. The game loop, combat HUD, inventory management, city facilities, and character creation all rely on terminal ASCII rendering. Migrating to a GUI requires a **layered approach** that keeps game logic intact while replacing the presentation layer.

Recommended approach: Hybrid Adapter + Screen-Based GUI using **tkinter** for the initial GUI (built-in, no extra dependencies, cross-platform). A future upgrade path to **Pygame** is documented for advanced visuals.

2. Current Architecture Analysis

2.1 Entry Flow

```

main.py
├─ main_menu()           ← Terminal menu (New Game / Continue / Delete / Quit)
├─ create_character()    ← Race/class selection, point allocation, name input
├─ play_game()
│   ├─ visit_city()      ← City hub loop → facilities
│   └─ explore_dungeon() ← Dungeon crawl loop
│       └─ combat()       ← Turn-based combat engine

```

2.2 I/O Dependency Matrix

Every file that directly calls `print()`, `input()`, or `clear_screen()` must be touched:

Category	Files	I/O Density
Core Loop	main.py, city.py, dungeon.py	High
Combat	combat/combat_engine.py, combat/combat_ui.py, combat/player_actions.py, combat/ally.py, combat/superboss_common.py + 7 superboss files	Very High
Facilities	facilities/shop.py, facilities/inn.py, facilities/blacksmith.py, facilities/temple.py, facilities/guild.py, facilities/house.py, facilities/barracks.py, facilities/gift_shop.py, facilities/travel.py, facilities/travel_events.py, facilities/port.py, facilities/shipyard.py, facilities/trade_hall.py, facilities/herbalist.py, facilities/arcane_tower.py, facilities/black_market.py, facilities/skill_book.py	High
Inventory	inventory_ui.py, inventory.py	Medium
Character	character.py	Medium
Utility	utils.py, events.py, dungeon_rooms.py, save_load.py, city_dialogue.py	Low-Medium

2.3 Current HUD Systems

- Combat HUD** (`combat/combat_ui.py`): Fixed-width ASCII box (68 chars inner). Two-column layout. HP bars, buff tags, action menu, cooldown display.
- City HUD** (`city.py`): Header with name, floor progress, date/time, gold, mount, status effects.
- Inventory HUD** (`inventory_ui.py`): Character stats with equipment/buff breakdown, ally stats, skill lists, bounty tracking.
- Dungeon HUD** (`dungeon_rooms.py`): ASCII map, room labels, post-room menu.

3. Framework Decision

3.1 Why tkinter for Phase 1–2

Criterion	tkinter	Pygame	Dear PyGui	PyQt6
Built-in	✔ Yes	✗ pip install	✗ pip install	✗ pip install
EXE bundling	✔ Easy	⚠ Heavy	⚠ Binary deps	⚠ Large size
ASCII art	✔ Text widget	✗ Need rewrite	⚠ Text-only	✔ Text widget
Menu/forms	✔ Good	⚠ Manual	✔ Good	✔ Excellent
Animation	✗ No	✔ Yes	✔ Yes	⚠ Limited
Game loop	⚠ Event-driven	✔ Native	✔ Native	⚠ Event-driven
Learning curve	Low	Medium	Low	Medium-High

Verdict: tkinter is the safest choice. It ships with Python, requires zero install steps, and `make_exe.py` already works with it. The ASCII art can be rendered in a `tk.Text` widget with a monospace font. Buttons, menus, and forms map cleanly.

3.2 Pygame Upgrade Path (Phase 3+)

If you want animated combat, particle effects, or sprite-based enemy portraits later, a **Pygame renderer** can be written as an alternative frontend. The game logic would remain identical — only the `gui/` presentation layer would change. This is documented but not required for the initial migration.

4. Migration Strategy

4.1 Core Principle: Game Logic is Pure

DO NOT rewrite `combat_engine.py`, `dungeon.py`, `leveling.py`, `events.py`, or any data files. The migration is **presentation-layer only**.

4.2 The Three Strategies Compared

Strategy	Description	Effort	Risk
A. Big Bang Rewrite	Rewrite every file's print/input to GUI widgets	6–8 weeks	Very High — easy to break game logic
B. Adapter Pattern	Create a <code>gui/io_adapter.py</code> that redirects print/input to GUI	2–3 weeks	Medium — limited visual quality
C. Screen-Based Hybrid <i>(Recommended)</i>	Create <code>gui/screens/</code> with dedicated widgets per screen; refactor I/O in phases	4–6 weeks	Low-Medium — gradual, testable

4.3 Recommended: Screen-Based Hybrid

We create a `gui/` package with screen controllers. Each major game state gets its own screen class. Game functions are **lightly refactored** to return data instead of printing it, while the screen class renders it.

Example transformation:

```
# BEFORE (terminal):
def main_menu():
    while True:
        clear_screen()
        print("1. New Game")
        print("2. Continue")
        choice = input("Enter choice: ").strip()
        ...

# AFTER (GUI):
def main_menu():
    """Returns the player's choice as a string. No print/input."""
    return gui.get_main_menu_choice() # Blocks until user clicks a button
```

5. Files to Change — Complete Inventory

● Critical — Must Be Refactored (direct print/input/clear_screen)

#	File	Lines	What It Does	GUI Migration
1	main.py	218	Main menu, save/load menu, post-floor menu, game loop	Full refactor to <code>gui/screens/main_menu_screen.py</code> + <code>gui/screens/post_floor_screen.py</code>
2	character.py	280	Character creation flow, point allocation, ensure_player_fields	Refactor creation flow to <code>gui/screens/char_create_screen.py</code> ; keep <code>ensure_player_fields()</code>
3	city.py	230	City hub menu, service routing, dungeon entry	Refactor to <code>gui/screens/city_screen.py</code>
4	dungeon.py	641	Dungeon crawl loop, room traversal, superboss encounters	Refactor to <code>gui/screens/dungeon_screen.py</code> ; keep floor generation logic
5	combat/combat_engine.py	405	Main combat loop, initiative, round flow	Refactor to <code>gui/screens/combat_screen.py</code> ; keep <code>_combat_inner()</code> logic but redirect all prints
6	combat/combat_ui.py	308	Combat HUD: ASCII boxes, HP bars, buff tags, action menu	Full rewrite to <code>gui/combat/combat_renderer.py</code>
7	combat/player_actions.py	423	Player turn handler: attack, skills, items, flee, capture	Refactor to return action results; UI handled by combat screen

8	<code>combat/ally.py</code>	697	Ally creation, ally turn handler, equip/unequip	Refactor ally turn to return actions; keep creation logic
9	<code>inventory_ui.py</code>	518	Inventory/stats display, equipment management, bag submenu	Full rewrite to <code>gui/screens/inventory_screen.py</code>
10	<code>dungeon_rooms.py</code>	~400	Non-combat rooms: fountain, merchant, trap, treasure, stat-check, ASCII map	Refactor each room handler to return events; render in dungeon screen
11	<code>utils.py</code>	151	<code>clear_screen()</code> , <code>format_time()</code> , <code>advance_time()</code> , <code>handle_player_death()</code>	Replace <code>clear_screen()</code> with GUI screen transitions; keep time logic

● High — Facility Files (each has menus, print/input)

#	File	GUI Migration
12	<code>facilities/shop.py</code>	<code>gui/screens/facilities/shop_screen.py</code>
13	<code>facilities/inn.py</code>	<code>gui/screens/facilities/inn_screen.py</code>
14	<code>facilities/blacksmith.py</code>	<code>gui/screens/facilities/blacksmith_screen.py</code>
15	<code>facilities/temple.py</code>	<code>gui/screens/facilities/temple_screen.py</code>
16	<code>facilities/guild.py</code>	<code>gui/screens/facilities/guild_screen.py</code>
17	<code>facilities/house.py</code>	<code>gui/screens/facilities/house_screen.py</code> — most complex facility
18	<code>facilities/barracks.py</code>	<code>gui/screens/facilities/barracks_screen.py</code>
19	<code>facilities/gift_shop.py</code>	<code>gui/screens/facilities/gift_shop_screen.py</code>
20	<code>facilities/travel.py</code>	<code>gui/screens/facilities/travel_screen.py</code>
21	<code>facilities/travel_events.py</code>	Integrate into travel screen
22	<code>facilities/port.py</code>	<code>gui/screens/facilities/port_screen.py</code>
23	<code>facilities/shipyard.py</code>	<code>gui/screens/facilities/shipyard_screen.py</code>
24	<code>facilities/trade_hall.py</code>	<code>gui/screens/facilities/trade_hall_screen.py</code>
25	<code>facilities/herbalist.py</code>	<code>gui/screens/facilities/herbalist_screen.py</code>
26	<code>facilities/arcane_tower.py</code>	<code>gui/screens/facilities/arcane_tower_screen.py</code>
27	<code>facilities/black_market.py</code>	<code>gui/screens/facilities/black_market_screen.py</code>
28	<code>facilities/skill_book.py</code>	<code>gui/screens/facilities/skill_book_screen.py</code>

● Medium — Combat Modules (partial refactor needed)

#	File	GUI Migration
29	combat/action_menu.py	Keep logic; GUI reads actions from it to build buttons
30	combat/superboss_common.py	Refactor print/input; keep loop logic
31	combat/broodmother.py	Refactor print/input; keep hook signatures
32	combat/slitcurrent.py	Same as above
33	combat/sylvana.py	Same as above
34	combat/ignis.py	Same as above
35	combat/yinglong.py	Same as above
36	combat/rientrante.py	Same as above
37	combat/everlong_ship.py	Same as above

● Low — Data & Logic Files (minimal or no changes)

#	File	Change
38	inventory.py	Remove print statements if any; keep pure logic
39	save_load.py	No changes needed — file I/O, not terminal I/O
40	leveling.py	No changes — pure logic
41	events.py	Replace <code>print()</code> alerts with GUI notification queue
42	city_dialogue.py	Replace <code>print()</code> with text display
43	combat/stats.py	No changes — pure logic
44	combat/skills.py	No changes — pure logic
45	combat/enemy_ai.py	No changes — pure logic
46	combat/elemental.py	No changes — pure logic
47	combat/status_effects.py	No changes — pure logic
48	combat/capture.py	Remove print statements
49	combat/stat_milestones.py	No changes — pure logic
50	resources/ (all)	No changes — data files

● Build & Config

#	File	Change
---	------	--------

51	make_exe.py	Add gui/ to PyInstaller --add-data ; add tkinter hooks
52	.gitignore	Add gui/__pycache__/ if needed

6. New Files to Create

6.1 Core GUI Framework

```
gui/
├─ __init__.py
├─ app.py           ← Main tkinter application class, window setup
├─ theme.py         ← Color scheme, fonts, styling constants
├─ screen_manager.py ← Handles screen transitions, back-stack
├─ notification_queue.py ← In-game alert/toast system (replaces event prints)
├─ io_redirector.py ← Optional: captures stray print() for debug window
├─ screens/
│   ├─ __init__.py
│   ├─ base_screen.py ← Abstract base: all screens inherit from this
│   ├─ main_menu_screen.py ← New Game, Continue, Delete Save, Quit
│   ├─ char_create_screen.py ← Race, class, point allocation, name, slot
│   ├─ city_screen.py ← City hub with service buttons
│   ├─ dungeon_screen.py ← Dungeon crawl: room transitions, ASCII map, post-room menu
│   ├─ combat_screen.py ← Combat HUD: party vs enemies, action buttons, turn order
│   ├─ inventory_screen.py ← Stats, equipment, bag, skill book (tabbed interface)
│   ├─ post_floor_screen.py ← Floor cleared: continue, return to city, save & quit
│   └─ facilities/
│       ├─ __init__.py
│       ├─ base_facility_screen.py
│       ├─ shop_screen.py
│       ├─ inn_screen.py
│       ├─ blacksmith_screen.py
│       ├─ temple_screen.py
│       ├─ guild_screen.py
│       ├─ house_screen.py
│       ├─ barracks_screen.py
│       ├─ gift_shop_screen.py
│       ├─ travel_screen.py
│       ├─ port_screen.py
│       ├─ shipyard_screen.py
│       ├─ trade_hall_screen.py
│       ├─ herbalist_screen.py
│       ├─ arcane_tower_screen.py
│       ├─ black_market_screen.py
│       └─ skill_book_screen.py
```

6.2 Combat GUI Subsystem

```
gui/combat/
├─ __init__.py
├─ combat_renderer.py      ← Renders party/enemy columns, HP bars, buff tags
├─ action_panel.py        ← Action buttons: Attack, Defend, Skills, Items, Flee, Capture
├─ initiative_bar.py      ← Turn order display
├─ damage_popup.py        ← Floating damage numbers (optional Phase 2)
├─ enemy_portrait.py      ← Enemy sprite placeholder + HP bar
```

6.3 Widget Library

```
gui/widgets/
├─ __init__.py
├─ hp_bar.py              ← Visual HP bar widget (ttk.Progressbar or custom canvas)
├─ buff_icon.py          ← Small colored badges for buffs/debuffs
├─ stat_display.py       ← Attribute line: `STR: 8 + 2(eq) + 1(buff) = 11`
├─ item_card.py          ← Equipment/item display with rarity color
├─ dialogue_box.py       ← Typewriter-effect text display for NPC dialogue
├─ confirm_dialog.py     ← Yes/No, OK/Cancel reusable dialogs
```

6.4 Entry Point

```
launcher.py      ← New! Chooses terminal or GUI mode
```

7. Implementation Phases & Timeline

Overview

Phase	Duration	Focus	Working Game?
Phase 0	Week 0	Setup, framework, architecture	No
Phase 1	Weeks 1–2	Main menu, character creation, city hub	Yes (menu + city only)
Phase 2	Weeks 3–4	Combat HUD, combat actions	Yes (full combat)
Phase 3	Weeks 5–6	Inventory, facilities, dungeon crawl	Yes (complete game)
Phase 4	Week 7+	Polish, animations, settings, testing	Yes (polished)

Weekly Breakdown

Week 0 — Foundation (Phase 0)

- ☐ Create `gui/` package structure
- ☐ Implement `gui/app.py` — tkinter window (900×700 default, resizable)
- ☐ Implement `gui/theme.py` — dark theme colors, fonts (JetBrains Mono or Consolas for ASCII)
- ☐ Implement `gui/screen_manager.py` — screen switching with transitions

- ☐ Implement `gui/widgets/` — HP bar, stat display, item card, confirm dialog
- ☐ Create `launcher.py` — `python launcher.py --gui` vs `python launcher.py --terminal`
- ☐ Update `make_exe.py` to include `gui/` and set GUI as default

Week 1 — Main Menu & Character Creation (Phase 1)

- ☐ `gui/screens/main_menu_screen.py` — New Game, Continue, Delete Save, Quit buttons
- ☐ Refactor `main.py` `main_menu()` to work in both terminal and GUI mode
- ☐ `gui/screens/char_create_screen.py`:
 - Race selection (cards with descriptions)
 - Class selection (cards with descriptions)
 - Point allocation (sliders or +/- buttons per attribute)
 - Name input (Entry widget)
 - Save slot selection (dropdown or grid)
- ☐ Refactor `character.py` `create_character()` and `allocate_points()`

Week 2 — City Hub (Phase 1 continued)

- ☐ `gui/screens/city_screen.py`:
 - Top status bar (name, floor, date, time, gold, mount, status)
 - Service buttons grid (Shop, Inn, Blacksmith, etc.)
 - House button (conditional)
 - Bottom action bar (Inventory, Skill Book, Travel, Dungeon, Save)
- ☐ Refactor `city.py` `visit_city()` — extract pure logic from print/input
- ☐ Integrate `events.py` with GUI notification queue

Week 3 — Combat Foundation (Phase 2)

- ☐ `gui/combat/combat_renderer.py`:
 - Left column: Player + allies (portrait placeholder, name, HP bar, buff icons)
 - Right column: Enemies (same layout)
 - ASCII art preservation in a Text widget with monospace font
- ☐ `gui/combat/action_panel.py`:
 - Dynamic action buttons from `combat/action_menu.py`
 - Skill buttons with cooldown gray-out
 - Item submenu popup
 - Target selection highlighting
- ☐ Refactor `combat/combat_ui.py` — all `print_*` functions become data-returning

Week 4 — Combat Polish & Ally Turns (Phase 2 continued)

- ☐ `gui/combat/initiative_bar.py` — Visual turn order strip
- ☐ Refactor `combat/player_actions.py` — `handle_player_turn()` returns actions, not prints
- ☐ Refactor `combat/ally.py` — `handle_ally_turn()` returns actions
- ☐ Integrate superboss fights (`combat/superboss_common.py` + 7 boss modules)

- ☐ Death screen (`gui/screens/death_screen.py`)

Week 5 — Inventory & Facilities (Phase 3)

- ☐ `gui/screens/inventory_screen.py` (tabbed notebook widget):
 - Tab 1: Stats & Equipment (equip/unequip with drag-and-drop or buttons)
 - Tab 2: Bag (use/drop items, stack counts)
 - Tab 3: Skill Book
 - Tab 4: Allies (recruit, dismiss, equipment)
- ☐ Refactor `inventory_ui.py` completely
- ☐ Facility screens: `shop_screen.py` , `inn_screen.py` , `blacksmith_screen.py`
- ☐ Facility screens: `temple_screen.py` , `guild_screen.py` , `barracks_screen.py`

Week 6 — House, Travel & Dungeon (Phase 3 continued)

- ☐ `gui/screens/facilities/house_screen.py` — Monster girl grid, affection bars, talk/gift/recruit
- ☐ `gui/screens/facilities/travel_screen.py` — City selection, travel events
- ☐ `gui/screens/dungeon_screen.py` :
 - Room transition animations (fade or slide)
 - ASCII map rendered in canvas or text widget
 - Post-room menu buttons
- ☐ Refactor `dungeon.py` `explore_dungeon()` — pure loop logic
- ☐ Refactor `dungeon_rooms.py` — return room results instead of printing

Week 7+ — Polish & Release (Phase 4)

- ☐ Settings screen (sound toggle, font size, color theme)
- ☐ Log window (scrollable combat/dungeon history)
- ☐ Keyboard shortcuts (1-9 for skills, A for attack, D for defend, etc.)
- ☐ Splash screen / title art
- ☐ Full playthrough testing
- ☐ Update `make_exe.py` and build release

8. Detailed Phase Breakdown

8.1 Phase 0: Architecture Decisions

Screen Manager Pattern

```
# gui/screen_manager.py
class ScreenManager:
    def __init__(self, root_window):
        self.root = root_window
        self.current_screen = None
        self.player = None # Shared game state

    def switch_to(self, screen_class, **kwargs):
        if self.current_screen:
            self.current_screen.destroy()
        self.current_screen = screen_class(self.root, self, **kwargs)
        self.current_screen.pack(fill="both", expand=True)
```

Base Screen Contract

```
# gui/screens/base_screen.py
import tkinter as tk
from abc import ABC, abstractmethod

class BaseScreen(tk.Frame, ABC):
    def __init__(self, parent, screen_manager, **kwargs):
        super().__init__(parent, bg=Theme.BG_DARK)
        self.sm = screen_manager
        self.player = screen_manager.player
        self.build_ui()

    @abstractmethod
    def build_ui(self):
        """Create all widgets for this screen."""
        pass
```

I/O Abstraction Strategy

Instead of rewriting every `print()` and `input()`, create an `IOWrapper` that the game logic uses:

```
# gui/io_adapter.py (optional, for gradual migration)
class GUIAdapter:
    def __init__(self, screen):
        self.screen = screen

    def print(self, text):
        self.screen.append_log(text)

    def input(self, prompt):
        return self.screen.wait_for_input(prompt)

    def clear(self):
        self.screen.clear_log()
```

Recommended: Skip the adapter. Refactor functions to **return data** and let the screen render it. This is cleaner long-term.

8.2 Phase 1: Menu & City (Detailed)

Main Menu Screen

```
+-----+
|          THRONE OF PLAINTEXT          |
|                                         |
|   [ New Game ]   [ Continue ]         |
|                                         |
|   [Delete Save]   [   Quit   ]         |
+-----+
```

Changes to `main.py`:

- Extract `main_menu()` loop body into a data structure
- `main_menu()` becomes:

```
def get_main_menu_actions():
    return [
        ("new", "New Game", lambda: create_character()),
        ("load", "Continue", lambda: show_load_dialog()),
        ...
    ]
```

Character Creation Screen

```

+-----+
| Step 1/4: Choose Race                               |
| [Human] [Elf] [Dwarf] [Orc] [Beastkin] ...         |
| Description panel updates on hover/click           |
+-----+
| Step 2/4: Choose Class                             |
| [Warrior] [Mage] [Rogue] [Cleric] ...              |
+-----+
| Step 3/4: Allocate Points (28 remaining)           |
| STR [---●-----] 8   DEX [-----●---] 5          |
| CON [---●-----] 8   LRN [-----●---] 5          |
| WIS [-----●---] 5   CHA [-----●---] 5          |
+-----+
| Step 4/4: Name & Save Slot                         |
| Name: [_____] Slot: [1 ▼]                          |
|           [ Create Character ]                      |
+-----+

```

Changes to `character.py`:

- `allocate_points()` becomes `validate_point_allocation(base_attrs, 分配)` — returns final attrs or error
- `create_character()` becomes `build_character(race_key, class_key, attrs, name, slot)` — returns player dict
- The UI handles the interactive loop

City Hub Screen

```

+-----+
| Solmere | Floor 12/45 | Day 14 | 14:30 | 1,240g |
+-----+
| [Shop] [Inn] [Blacksmith] [Temple] [Guild]         |
| [Port] [Shipyard] [Trade] [Barracks] [Herbal]      |
| [Arcane] [Black Market] [Gift Shop]                |
|                                                       |
|           [Your House (Manor)]                      |
+-----+
| [View Stats] [Skill Book] [Travel] [Dungeon]       |
|           [Save & Quit]                             |
+-----+

```

Changes to `city.py`:

- `visit_city()` loop body becomes `get_city_state(player, city_id)` → returns menu data
- Service handlers return result dicts instead of printing
- Event alerts go to `gui/notification_queue.py`

8.3 Phase 2: Combat (Detailed)

Combat Screen Layout

```
+-----+
| >> ROUND 3 | Floor 7 | Room 4/10 | Time: 14:30 |
+-----+-----+
| YOUR PARTY          | ENEMIES          |
| -----|-----|
| * You    106/120 [#####..] | [1] Goblin      |
|   [+STR][DEF]          |   24/30 [###] |
|                       |   [SLW]       |
| [2] Lyra  89/95  [#####.#] | [2] Wolf        |
|   [REG][BLS]          |   18/25 [##]  |
|                       |   [PSN]       |
|                       | [3] Empowered  |
|                       |   Orc  55/60  |
+-----+-----+
| ✂ ROUND 3: ACTION PHASE |
| -----|
|
| [Attack] [Defend] [Use Item] [Flee] [Capture] |
| [1] Slash [2] Power Strike [3] Berserk      |
|
| (Crew Rally recharging: 2 turn(s)) |
+-----+-----+
| Log: Lyra pierces Wolf for 12 damage! |
|      Wolf is bleeding!                |
+-----+-----+
```

Key Implementation Notes:

- The ASCII box is rendered in a `tk.Text` widget with monospace font
- Action buttons are `tk.Button` widgets that call `handle_player_turn()` with the action
- Skill buttons are dynamically generated from `combat/action_menu.py`
- HP bars are `tk.Canvas` or `ttk.Progressbar` widgets
- Buff tags are small colored `tk.Label` widgets
- The log at the bottom is a read-only `tk.Text` with scrollbar

Critical Refactor: `combat/combat_ui.py`

Every `print_*` function becomes a data formatter:

```
# BEFORE:
def print_combat_hud(player, enemies, active_ally=None, header=""):
    print("+" + "-" * 68 + "+")
    ...

# AFTER:
def format_combat_hud_data(player, enemies, active_ally=None, header=""):
    """Returns a dict with all HUD data. GUI renders it."""
    return {
        "party": format_party_data(player, active_ally),
        "enemies": format_enemy_data(enemies),
        "header": header,
        "action_menu": get_action_menu(player, enemies) if not active_ally else
        _ally_action_menu(active_ally, player, enemies),
    }
```

Critical Refactor: `combat/player_actions.py`

`handle_player_turn()` currently blocks on `input()`. In GUI mode, it must yield:

```
# APPROACH: Coroutine / State Machine
class CombatStateMachine:
    def __init__(self, player, enemies):
        self.player = player
        self.enemies = enemies
        self.state = "waiting_for_action"
        self.pending_action = None

    def on_player_action(self, action):
        """Called by GUI when player clicks a button."""
        self.pending_action = action
        self.state = "processing"
        self.tick() # Resume combat logic

    def tick(self):
        # ... combat logic continues ...
```

8.4 Phase 3: Inventory & Facilities (Detailed)

Inventory Screen (Tabbed)

```

+-----+
| [Stats/Equip] [Bag] [Skills] [Allies] [Bounties]|
+-----+
| === Arin (Level 12) ===|
| HP: 106/120 | Gold: 1,240|
| | |
| STR: 8 + 2 = 10 [Milestone: +1 flat dmg]|
| DEX: 5 + 1 = 6|
| ...|
| |
| Equipment:|
| [Weapon] Iron Sword [rare] | ATK +5|
| [Armor] Leather Vest [unc] | CON +2|
| [Acc 1] Ring of Vigor [epic] | STR +3|
| [Acc 2] (empty)|
| |
| [Equip Item] [Unequip Slot]|
+-----+

```

Facility Screens

Most facilities follow a simple pattern:

1. **Header** — facility name, player gold
2. **Content area** — item list, NPC dialogue, or interactive grid
3. **Action bar** — Buy, Sell, Upgrade, Back buttons

The `facilities/` module files should be refactored to return data:

```

# BEFORE (facilities/shop.py):
def city_shop(player, city_id):
    while True:
        print("Welcome to the shop!")
        for idx, item in enumerate(stock):
            print(f"{idx+1}. {item['name']} — {item['price']}g")
        choice = input("Buy which? (0 to leave): ")
        ...

# AFTER:
def get_shop_stock(player, city_id):
    """Returns list of available items with prices."""
    return [...]

def buy_item(player, city_id, item_index):
    """Returns (success, message, updated_player)."""
    ...

```

8.5 House Screen (Most Complex Facility)

The house has multiple subsystems:

- Monster girl storage (grid of portraits/cards)
- Affection system (progress bars)
- Talk / Gift / Recruit actions (daily limits)
- Engagement / Wedding (ring consumption, soulbound items)
- Lounge with ally ascension menu

```
+-----+
| Your House (Manor) | Income: 340g ready |
+-----+
| [Collect Income] [Rest] [Sleep] [Back] |
+-----+
| Monster Girls (12/20): |
| +-----+ +-----+ +-----+ +-----+ |
| | Lyra | | Mira | | Syl | | Kora | |
| | ♀ 78♥ | | ♀ 45♥ | | ♀ 100♥ | | ♀ 12♥ | |
| +-----+ +-----+ +-----+ +-----+ |
| |
| [Selected: Lyra] |
| Affection: [██████████████████████] 78/100 |
| [Talk] [Gift] [Recruit] [Propose] |
+-----+
```

9. Key Design Decisions

9.1 Terminal Mode Preservation

Decision: Keep terminal mode fully functional. Add a `--terminal` flag to `launcher.py`.

Rationale:

- Faster debugging during GUI development
- Fallback if GUI has bugs
- Some players prefer terminal aesthetic
- CI/testing can run headless

```
# launcher.py
import sys

from main import main_menu as terminal_main
from gui.app import run_gui

if __name__ == "__main__":
    if "--terminal" in sys.argv:
        terminal_main()
    else:
        run_gui()
```

9.2 Monospace Font for ASCII Art

Decision: Use `Consolas` (Windows) / `DejaVu Sans Mono` (Linux) / `Menlo` (macOS) at 11–12pt.

Rationale: The combat HUD, dungeon map, and many screens rely on fixed-width alignment. A proportional font will break the layout.

```
# gui/theme.py
import platform

FONT_FAMILY = {
    "Windows": "Consolas",
    "Darwin": "Menlo",
    "Linux": "DejaVu Sans Mono",
}.get(platform.system(), "Courier")

FONT_SIZE = 11
FONT = (FONT_FAMILY, FONT_SIZE)
FONT_BOLD = (FONT_FAMILY, FONT_SIZE, "bold")
```

9.3 Color Scheme (Dark Theme)

```
# gui/theme.py
class Theme:
    BG_DARK = "#1a1a2e"
    BG_MID = "#16213e"
    BG_LIGHT = "#0f3460"
    ACCENT = "#e94560"
    TEXT = "#eaeaea"
    TEXT_DIM = "#a0a0a0"
    HP_BAR_FILL = "#4ecca3"
    HP_BAR_EMPTY = "#2d3436"
    RARITY_COLORS = {
        "common": "#b0b0b0",
        "uncommon": "#1abc9c",
        "rare": "#3498db",
        "epic": "#9b59b6",
        "legendary": "#f1c40f",
    }
```

9.4 Keyboard Shortcuts

Key	Action
1 – 9	Skill slot (combat)
A	Attack
D	Defend
F	Flee
C	Capture
U	Use Item
I	Open Inventory (dungeon/city)
Enter	Confirm / Continue
Esc	Back / Cancel

9.5 Window Size & Responsiveness

Decision: Fixed minimum size 900×700, resizable. Use `tk.Grid` for layout (not `tk.Place`).

Rationale: tkinter's `grid()` system handles resizing cleanly. Combat HUD panels should expand proportionally.

10. Save File Compatibility

Decision: Save files remain **100% unchanged**. The `save_load.py` pickle format is identical.

Rationale:

- Zero migration risk for existing players
- GUI and terminal modes share the same save files
- No data loss

New fields needed for GUI-only settings:

```
# gui/settings.py (saved separately, not in player dict)
GUI_SETTINGS_FILE = "savefile/gui_settings.json"

def load_gui_settings():
    return {
        "window_size": "900x700",
        "font_size": 11,
        "theme": "dark",
        "sound_enabled": True,
        "keyboard_mode": False, # True = vim-style hjkl navigation
    }
```

11. Build System Updates

11.1 `make_exe.py` Changes

```
# Add to collect_data_files() if not already present:
gui_data = []
for root, dirs, files in os.walk("gui"):
    for f in files:
        if f.endswith(".py") or f.endswith(".json"):
            full = os.path.join(root, f)
            gui_data.append((full, root))

data_files.extend(gui_data)

# Ensure tkinter is included:
hiddenimports.extend([
    "tkinter",
    "tkinter.ttk",
    "tkinter.font",
    "tkinter.messagebox",
])
```

11.2 Entry Point

```
# In make_exe.py, change:
pyinstaller_args = [
    "main.py", # OLD
    # ...
]

# To:
pyinstaller_args = [
    "launcher.py", # NEW – default is GUI mode
    # ...
]
```

12. Testing Strategy

12.1 Unit Tests for GUI Widgets

```
# tests/gui/test_hp_bar.py
import unittest
from gui.widgets.hp_bar import HPBar

class TestHPBar(unittest.TestCase):
    def test_full_hp(self):
        bar = HPBar(None, 100, 100)
        self.assertEqual(bar.get_percentage(), 100.0)

    def test_half_hp(self):
        bar = HPBar(None, 50, 100)
        self.assertEqual(bar.get_percentage(), 50.0)
```

12.2 Integration Tests (Critical Paths)

#	Test	Command
1	Create character → enter city → enter dungeon	<code>python launcher.py</code>
2	Fight combat → use skill → win	GUI combat screen
3	Equip item → unequip → verify stats	Inventory screen
4	Save game → quit → load → verify state	Save/load flow
5	Terminal mode still works	<code>python launcher.py --terminal</code>
6	Old save loads in GUI	Copy old save, launch GUI

12.3 Regression Tests

After each phase, run a full dungeon floor (combat + non-combat rooms) in **both** terminal and GUI mode and compare:

- HP values match
- Gold/exp gains match
- Item drops match
- Save file is byte-identical after both runs

13. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Combat timing/feel degraded	Medium	High	Preserve ASCII art in Text widget; keep instant response (no animations in Phase 1)
tkinter looks dated	High	Low	Custom dark theme + monospace font makes it look intentional; upgrade to Pygame later if desired
Refactor introduces bugs	Medium	High	Keep terminal mode working; run both modes in parallel; add regression tests
Scope creep (facility overload)	High	Medium	Phase 1 & 2 first; some facilities can use generic "list + buttons" template
EXE size increase	Low	Low	tkinter is lightweight; PyInstaller handles it natively
Platform differences	Medium	Low	Test on Windows (primary), macOS, Linux; use platform-specific font fallbacks
Keyboard-only players inconvenienced	Medium	Medium	Full keyboard shortcut support; tab navigation

Appendix A: I/O Call Audit

Quick-reference count of `print()` and `input()` calls per file (approximate):

File	<code>print()</code>	<code>input()</code>	Notes
<code>main.py</code>	25	6	
<code>character.py</code>	20	8	
<code>city.py</code>	18	2	
<code>dungeon.py</code>	35	3	
<code>combat/combat_engine.py</code>	30	2	
<code>combat/combat_ui.py</code>	25	0	All prints
<code>combat/player_actions.py</code>	35	8	

combat/ally.py	30	6	
inventory_ui.py	40	6	
dungeon_rooms.py	25	3	
utils.py	12	1	handle_player_death()
facilities/shop.py	15	2	
facilities/inn.py	10	1	
facilities/blacksmith.py	18	3	
facilities/temple.py	12	2	
facilities/guild.py	15	2	
facilities/house.py	40	5	Most prints of any facility
facilities/barracks.py	12	2	
facilities/gift_shop.py	10	2	
facilities/travel.py	15	2	
facilities/travel_events.py	12	2	
facilities/*.py (remaining)	~60 total	~10 total	
TOTAL	~534	~78	

Conclusion: ~534 `print()` calls and ~78 `input()` calls must be addressed. The majority are in combat and facilities.

Appendix B: GUI Wireframe Notes

B.1 Screen Transition Flow

[Main Menu]

|— New Game → [Character Creation] → [City Hub]
|— Continue → [Load Save] → [City Hub] or [Dungeon]
└— Quit

[City Hub]

|— Services → [Facility Screens] → [City Hub]
|— House → [House Screen] → [City Hub]
|— Inventory → [Inventory Screen] → [City Hub]
|— Travel → [Travel Screen] → [City Hub] (new city)
|— Dungeon → [Dungeon Screen]
└— Save & Quit → [Main Menu]

[Dungeon Screen]

|— Combat Room → [Combat Screen] → [Dungeon Screen]
|— Non-Combat Room → [Room Event UI] → [Dungeon Screen]
|— Inventory (I key) → [Inventory Screen] → [Dungeon Screen]
└— Boss Room → [Combat Screen] → [Post-Floor Screen]

[Combat Screen]

|— Player Turn → Action buttons active
|— Ally Turn → Action buttons for ally
|— Enemy Turn → Auto-play with log
|— Victory → Loot screen → [Dungeon Screen]
|— Flee → [City Hub]
└— Death → [Death Screen] → [City Hub] or [Main Menu]

B.2 Responsive Layout Strategy

Root Window (tk.Tk)

└— Main Frame (fills window)
 |— Top Bar (fixed height 40px): Status info
 |— Content Area (expands): Current screen
 └— Bottom Bar (fixed height 100px): Log / notifications

- All screens inherit from `BaseScreen` and fill the Content Area
- Top Bar shows: Name, Location, Floor, Day, Time, Gold — always visible
- Bottom Bar shows: Last 3 log lines + notification toasts

End of Document